



Introducción a la Inteligencia Artificial

Taller Investigativo

Profesor:

Ing. Julio Omar Palacio Niño, M.Sc.
palacio_julio@javeriana.edu.co

Temática:

Algoritmos de búsqueda local

Introducción:

Los algoritmos de búsqueda tradicionales se centran en encontrar una secuencia de pasos para alcanzar una solución. Sin embargo, en muchos problemas de optimización, el interés no reside en el camino, sino en encontrar una solución óptima sin importar los pasos requeridos. Para estos escenarios, se emplean técnicas de búsqueda local. Estas son fundamentales en aplicaciones como el diseño de circuitos integrados, optimización de redes, inteligencia artificial y gestión de recursos.

Temática

Este taller se enfoca en el estudio y aplicación de diversos algoritmos de búsqueda local. Cada grupo investigará a fondo uno de los siguientes algoritmos:

- Hill climbing (Ascensión de colina)
- Tabu Search (Búsqueda Tabú)
- Simulated Annealing (Recocido Simulado)
- Evolutionary Algorithms (Algoritmos Genéticos)
- Memetic Algorithm (Algoritmo Memético)
- Ant Colony Optimization (Colonia de Hormigas)
- Greedy Randomized Adaptive Search Procedure (GRASP)

Implementación

Para garantizar una comparación justa y directa, todos los grupos resolverán el mismo problema base: **El Problema de las N-Reinas Escalable**. El objetivo es ubicar N reinas en un tablero de $N \times N$ sin que se ataquen entre sí (filas, columnas o diagonales).

Todos los algoritmos deben construirse sobre el siguiente **marco de trabajo unificado**:

- **Representación del Estado:** Un vector unidimensional de tamaño N , donde el índice representa la columna y el valor representa la fila (ej. [2, 0, 3, 1]). Esto garantiza por defecto que no haya ataques en filas ni columnas.
- **Función de Costo (Objetivo):** Contar el número total de ataques en las diagonales. Una solución óptima tiene un costo exacto de 0.
- **Operador de Vecindario Base:** Realizar un *swap* (intercambio) entre los valores de dos posiciones aleatorias del vector para generar nuevas soluciones.

Consideraciones Específicas por Algoritmo

Dado que todos utilizarán la misma representación y problema base, cada grupo debe tener en cuenta las siguientes consideraciones técnicas para evitar bloqueos en su implementación:

- **Algoritmos Genéticos y Meméticos:** Al utilizar un vector donde no se pueden repetir números (para evitar ataques en filas), **no pueden usar el operador de cruce tradicional**. Cruzar dos vectores directamente generaría reinas repetidas. Deben investigar e implementar obligatoriamente operadores de cruce para permutaciones, como el *Cruce de Orden (OX - Order Crossover)* o el *PMX*.
- **GRASP:** En su fase constructiva, no deben empezar con el vector lleno al azar. Deben construir el tablero columna por columna. La *Lista Restringida de Candidatos (RCL)* debe formarse evaluando qué filas disponibles generan el menor número de ataques diagonales contra las reinas ya colocadas.
- **Búsqueda Tabú:** La Lista Tabú **no debe guardar el tablero completo** (lo cual colapsaría la memoria en tableros grandes), sino el movimiento específico que se hizo (ej. "prohibido intercambiar las posiciones 2 y 5") durante un número definido de iteraciones (*Tenencia Tabú*).
- **Recocido Simulado:** El éxito dependerá del esquema de enfriamiento. Se recomienda que la temperatura inicial sea calculada dinámicamente en función de los peores choques del tablero inicial, y no un número arbitrario, asegurando que el algoritmo acepte peores soluciones al inicio pero converja al final.
- **Colonia de Hormigas (ACO):** El espacio de búsqueda de las feromonas funciona distinto al vector. Las hormigas deben depositar feromonas en una matriz virtual de $N \times N$, donde la cantidad de feromona en la celda (i, j) dicta la probabilidad de que una hormiga decida poner la reina de la columna j en la fila i .

Entregables del taller:

La evaluación se basará en tres componentes: un artículo de investigación, una presentación oral y la implementación práctica.

3.1. Artículo de Investigación (Formato IEEE) Cada grupo deberá redactar un documento técnico sobre el algoritmo asignado. El artículo debe tener una extensión de entre 1500 y 2000 palabras, estar escrito en español y seguir estrictamente el formato IEEE. La estructura debe incluir:

- **Introducción:** Presentar el concepto de búsqueda local y la relevancia del algoritmo estudiado.
- **Concepto:** Explicar en detalle los fundamentos teóricos del algoritmo, su inspiración (ej. metalurgia, evolución, biología) y componentes clave.
- **Algoritmo:** Incluir el pseudocódigo y una descripción paso a paso de su funcionamiento.
- **Aplicaciones:** Describir cómo se resolvió el problema de las N-Reinas, qué representación se usó y analizar los resultados obtenidos ante diferentes tamaños del tablero.

La entrega final será en formato PDF a través de la plataforma virtual.

3.2. Presentación:

Adicionalmente se debe preparar una presentación del algoritmo para ser expuesto ante la clase, con una duración mínima de 15 minutos. Deben explicar claramente cómo adaptaron el algoritmo al problema de las N-Reinas y justificar las decisiones de diseño y selección de hiperparámetros.

3.3. Implementación Práctica en Python

El código debe cumplir con los siguientes requisitos técnicos:

- **Modularidad:** El código debe estar estructurado en funciones o clases claras, separando la evaluación de la función de costo, la generación de vecinos y el ciclo principal del algoritmo.
- **Escalabilidad:** El script debe recibir N (número de reinas) como parámetro. Debe probarse con el caso base de $N = 8$, pero debe ser capaz de procesar tableros más grandes (ej. $N = 50$ o $N = 100$) para evaluar el rendimiento computacional.
- **Ajuste de Parámetros:** Las variables clave (temperatura, iteraciones, tamaño de la lista tabú, factor de evaporación de feromonas, etc.) deben declararse al inicio para facilitar la experimentación.

- **Visualización y Métricas:** Usando librerías como Matplotlib o Seaborn (o equivalentes para consola), se debe imprimir:
 1. El tablero final obtenido o el vector solución.
 2. El costo final (número de ataques diagonales) y el tiempo de ejecución en segundos.
 3. Una gráfica de convergencia que muestre cómo disminuyó la función de costo iteración tras iteración.